```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

np.random.seed(42)
sns.set_theme(style="whitegrid")

OUTPUT_DIR = Path("/content/retail_analysis_output")
OUTPUT_DIR.mkdir(exist_ok=True)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
n = 150

products = {
    "Laptop": ("Electronics", 65000),
    "Phone": ("Electronics", 25000),
    "Tablet": ("Electronics", 18000),
    "Headphones": ("Accessories", 2000),
    "Mouse": ("Accessories", 500),
    "Keyboard": ("Accessories", 1500),
    "Chair": ("Furniture", 7000),
    "Desk": ("Furniture", 12000),
    "Notebook": ("Stationery", 100),
    "Pen": ("Stationery", 30)
}

product_names = list(products.keys())

df = pd.DataFrame({
    "Order_ID": range(1001, 1001 + n),
    "Order_Date": pd.date_range(
        "2025-01-01", periods=n, freq="2D"
    ).strftime("%Y-%m-%d"),
    "Product": np.random.choice(product_names, n),
    "Region": np.random.choice(
        ["North", "South", "East", "West"], n
    ),
    "Quantity": np.random.randint(1, 11, n)
})

df["Category"] = df["Product"].map(
    lambda p: products[p][0]
)
df["Unit_Price"] = df["Product"].map(
    lambda p: products[p][1]
).astype(float)

df["Unit_Price"] = (
    df["Unit_Price"] *
    np.random.choice([0.9, 1.0, 1.1], n)
).round(2)

# Introduce missing values
df.loc[[4, 18, 37], "Quantity"] = np.nan
df.loc[[8, 29, 51], "Region"] = np.nan
df.loc[[12, 42], "Category"] = np.nan

# Inconsistent category and region labels
df.loc[15, "Category"] = "electronics"
df.loc[26, "Category"] = " Accessories "
df.loc[32, "Region"] = "south"

# Mixed and invalid dates
df.loc[5, "Order_Date"] = "15/01/2025"
df.loc[10, "Order_Date"] = "31/02/2025"
df.loc[22, "Order_Date"] = "not-a-date"
df.loc[40, "Order_Date"] = None

# Extreme price for outlier detection
df.loc[60, "Unit_Price"] = 999999.0

# Add three duplicate records
df = pd.concat([df, df.iloc[[2, 8, 20]]], ignore_index=True)

df.to_csv(OUTPUT_DIR / "retail_sales_raw.csv", index=False)
```

```
print("Raw dataset shape:", df.shape)
display(df.head(10))
```

```
Raw dataset shape: (153, 7)
```

	Order_ID	Order_Date	Product	Region	Quantity	Category	Unit_Price
0	1001	2025-01-01	Chair	East	4.0	Furniture	6300.0
1	1002	2025-01-03	Headphones	East	6.0	Accessories	1800.0
2	1003	2025-01-05	Desk	South	8.0	Furniture	13200.0
3	1004	2025-01-07	Mouse	West	4.0	Accessories	550.0
4	1005	2025-01-09	Chair	East	NaN	Furniture	6300.0
5	1006	15/01/2025	Pen	North	9.0	Stationery	30.0
6	1007	2025-01-13	Tablet	South	3.0	Electronics	16200.0
7	1008	2025-01-15	Chair	North	9.0	Furniture	7700.0
8	1009	2025-01-17	Desk	NaN	2.0	Furniture	13200.0
9	1010	2025-01-19	Mouse	East	2.0	Accessories	500.0

```
print("Dataset shape:", df.shape)

df.info()

print("\nNumerical summary:")
display(df.describe())

print("\nMissing values:")
display(df.isna().sum())

print("\nDuplicate records:", df.duplicated().sum())
```

```
Dataset shape: (153, 7)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 153 entries, 0 to 152
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Order_ID    153 non-null    int64
 1   Order_Date  152 non-null    object
 2   Product     153 non-null    object
 3   Region      149 non-null    object
 4   Quantity    150 non-null    float64
 5   Category    151 non-null    object
 6   Unit_Price  153 non-null    float64
dtypes: float64(2), int64(1), object(4)
memory usage: 8.5+ KB

Numerical summary:
```

	Order_ID	Quantity	Unit_Price
count	153.000000	150.000000	153.000000
mean	1074.235294	5.400000	18421.732026
std	43.952840	2.937835	81865.817416
min	1001.000000	1.000000	27.000000
25%	1036.000000	3.000000	500.000000
50%	1074.000000	5.500000	6300.000000
75%	1112.000000	8.000000	16200.000000
max	1150.000000	10.000000	999999.000000

```
Missing values:
```

	0
Order_ID	0
Order_Date	1
Product	0
Region	4
Quantity	3
Category	2
Unit_Price	0

dtype: int64

```
Duplicate records: 3
```

```
clean_df = df.copy()

# Standardize column names
clean_df.columns = (
    clean_df.columns.str.strip()
    .str.lower()
    .str.replace(" ", "_")
)

# Remove duplicates
initial_rows = len(clean_df)
clean_df = clean_df.drop_duplicates()
duplicates_removed = initial_rows - len(clean_df)

# Standardize text columns
for col in ["product", "category", "region"]:
    clean_df[col] = clean_df[col].astype("string").str.strip()

clean_df["product"] = clean_df["product"].str.title()
clean_df["region"] = clean_df["region"].str.title()

# Correct categories using the product reference
product_to_category = {
    product: details[0]
    for product, details in products.items()
}
clean_df["category"] = clean_df["product"].map(product_to_category)

# Convert mixed date formats; invalid dates become NaT
clean_df["order_date"] = pd.to_datetime(
    clean_df["order_date"],
```

```
    format="mixed",
    dayfirst=True,
    errors="coerce"
)

# Convert numerical columns
clean_df["quantity"] = pd.to_numeric(
    clean_df["quantity"], errors="coerce"
)
clean_df["unit_price"] = pd.to_numeric(
    clean_df["unit_price"], errors="coerce"
)

# Remove rows with invalid or missing dates
invalid_dates = clean_df["order_date"].isna().sum()
clean_df = clean_df.dropna(subset=["order_date"])

# Handle invalid numerical values
clean_df.loc[clean_df["quantity"] <= 0, "quantity"] = np.nan
clean_df.loc[clean_df["unit_price"] <= 0, "unit_price"] = np.nan

# Fill missing numerical values using medians
clean_df["quantity"] = clean_df["quantity"].fillna(
    clean_df["quantity"].median()
)
clean_df["unit_price"] = clean_df["unit_price"].fillna(
    clean_df["unit_price"].median()
)

clean_df["quantity"] = clean_df["quantity"].round().astype(int)

# Fill missing regions using the most frequent region
clean_df["region"] = clean_df["region"].fillna(
    clean_df["region"].mode().iloc[0]
)

# Detect outliers using IQR
q1 = clean_df["unit_price"].quantile(0.25)
q3 = clean_df["unit_price"].quantile(0.75)
iqr = q3 - q1

lower_bound = max(0, q1 - 1.5 * iqr)
upper_bound = q3 + 1.5 * iqr

outlier_mask = (
    (clean_df["unit_price"] < lower_bound) |
    (clean_df["unit_price"] > upper_bound)
)
outliers_detected = int(outlier_mask.sum())

# Cap extreme prices instead of deleting the whole order
clean_df["unit_price"] = clean_df["unit_price"].clip(
    lower=lower_bound, upper=upper_bound
)

# Calculate sales and extract date components
clean_df["sales"] = (
    clean_df["quantity"] * clean_df["unit_price"]
).round(2)

clean_df["year"] = clean_df["order_date"].dt.year
clean_df["month"] = clean_df["order_date"].dt.month
clean_df["month_name"] = clean_df["order_date"].dt.strftime("%b")
clean_df["quarter"] = clean_df["order_date"].dt.quarter

clean_df = clean_df.sort_values("order_date").reset_index(drop=True)

clean_df.to_csv(
    OUTPUT_DIR / "retail_sales_cleaned.csv", index=False
)

print("Duplicates removed:", duplicates_removed)
print("Invalid/missing dates removed:", invalid_dates)
print("Price outliers detected:", outliers_detected)
print("IQR lower bound:", lower_bound)
print("IQR upper bound:", upper_bound)
print("Cleaned dataset shape:", clean_df.shape)

display(clean_df.head(10))
```

```
Duplicates removed: 3
Invalid/missing dates removed: 3
Price outliers detected: 14
IQR lower bound: 0
IQR upper bound: 39750.0
Cleaned dataset shape: (147, 12)
```

	order_id	order_date	product	region	quantity	category	unit_price	sales	year	month	month_name	quarter
0	1001	2025-01-01	Chair	East	4	Furniture	6300.0	25200.0	2025	1	Jan	1
1	1002	2025-01-03	Headphones	East	6	Accessories	1800.0	10800.0	2025	1	Jan	1
2	1003	2025-01-05	Desk	South	8	Furniture	13200.0	105600.0	2025	1	Jan	1
3	1004	2025-01-07	Mouse	West	4	Accessories	550.0	2200.0	2025	1	Jan	1
4	1005	2025-01-09	Chair	East	6	Furniture	6300.0	37800.0	2025	1	Jan	1
5	1007	2025-01-13	Tablet	South	3	Electronics	16200.0	48600.0	2025	1	Jan	1
6	1006	2025-01-15	Pen	North	9	Stationery	30.0	270.0	2025	1	Jan	1
7	1008	2025-01-15	Chair	North	9	Furniture	7700.0	69300.0	2025	1	Jan	1
8	1009	2025-01-17	Desk	West	2	Furniture	13200.0	26400.0	2025	1	Jan	1
9	1010	2025-01-19	Mouse	East	2	Accessories	500.0	1000.0	2025	1	Jan	1

```
print("Missing values:")
display(clean_df.isna().sum())

print("Duplicate rows:", clean_df.duplicated().sum())
print("\nData types:")
display(clean_df.dtypes)

print("\nDate range:")
print(clean_df["order_date"].min(), "to", clean_df["order_date"].max())

assert clean_df["order_date"].notna().all()
assert clean_df["quantity"].gt(0).all()
assert clean_df["unit_price"].gt(0).all()
assert clean_df["sales"].ge(0).all()
assert not clean_df.duplicated().any()

print("\nAll validation checks passed!")
```

```
Missing values:
```

	0
order_id	0
order_date	0
product	0
region	0
quantity	0
category	0
unit_price	0
sales	0
year	0
month	0
month_name	0
quarter	0

dtype: int64

```
Duplicate rows: 0

Data types:
```

	0
order_id	int64
order_date	datetime64[ns]
product	string[python]
region	string[python]
quantity	int64
category	object
unit_price	float64
sales	float64
year	int32
month	int32
month_name	object
quarter	int32

dtype: object

```
Date range:
2025-01-01 00:00:00 to 2025-10-26 00:00:00

All validation checks passed!
```

```
category_summary = clean_df.groupby("category").agg(
    total_sales=("sales", "sum"),
    order_count=("order_id", "count"),
    average_sales=("sales", "mean"),
    minimum_sales=("sales", "min"),
    maximum_sales=("sales", "max"),
    total_quantity=("quantity", "sum")
).round(2).sort_values("total_sales", ascending=False)

region_summary = clean_df.groupby("region").agg(
    total_sales=("sales", "sum"),
    order_count=("order_id", "count"),
    average_sales=("sales", "mean"),
    total_quantity=("quantity", "sum")
).round(2).sort_values("total_sales", ascending=False)

product_summary = clean_df.groupby("product").agg(
    total_sales=("sales", "sum"),
    units_sold=("quantity", "sum"),
    average_unit_price=("unit_price", "mean"),
    order_count=("order_id", "count")
).round(2).sort_values("total_sales", ascending=False)

monthly_summary = clean_df.groupby(
    clean_df["order_date"].dt.to_period("M")
).agg(
```

```
    total_sales=("sales", "sum"),
    order_count=("order_id", "count"),
    units_sold=("quantity", "sum")
).reset_index()

monthly_summary["order_date"] = (
    monthly_summary["order_date"].dt.to_timestamp()
)

print("Category summary")
display(category_summary)

print("Region summary")
display(region_summary)

print("Product summary")
display(product_summary)

print("Monthly summary")
display(monthly_summary)
```

Category summary

category	total_sales	order_count	average_sales	minimum_sales	maximum_sales	total_quantity
Electronics	6136900.0	40	153422.50	25000.0	397500.0	240
Furniture	1948600.0	38	51278.95	7700.0	108000.0	216
Accessories	318300.0	37	8602.70	500.0	79500.0	187
Stationery	11382.0	32	355.69	27.0	1100.0	155

Region summary

region	total_sales	order_count	average_sales	total_quantity
East	2887199.0	38	75978.92	223
North	2405750.0	32	75179.69	190
West	1784733.0	50	35694.66	252
South	1337500.0	27	49537.04	133

Product summary

product	total_sales	units_sold	average_unit_price	order_count
Laptop	2623500.0	66	39750.00	13
Phone	1780000.0	73	24166.67	12
Tablet	1733400.0	101	17040.00	15
Desk	1076400.0	92	11929.41	17
Chair	872200.0	124	7100.00	21
Headphones	216500.0	70	4903.85	13
Keyboard	66300.0	45	1466.67	9
Mouse	35500.0	72	496.67	15
Notebook	9720.0	97	100.59	17
Pen	1662.0	58	28.60	15

Monthly summary

	order_date	total_sales	order_count	units_sold
0	2025-01-01	579670.0	15	76
1	2025-02-01	735949.0	13	58
2	2025-03-01	467638.0	14	57
3	2025-04-01	709581.0	15	66
4	2025-05-01	1140532.0	16	96
5	2025-06-01	1791350.0	15	106
6	2025-07-01	771751.0	15	82
7	2025-08-01	1031681.0	16	88
8	2025-09-01	944885.0	15	91
9	2025-10-01	242145.0	13	78

Next steps: Generate code with `category_summary` | New interactive sheet | Generate code with `region_summary` | New interactive sheet | Ge

```
print("Highest-sales category:", category_summary["total_sales"].idxmax())
print("Lowest-sales category:", category_summary["total_sales"].idxmin())

print("Highest-sales region:", region_summary["total_sales"].idxmax())
print("Lowest-sales region:", region_summary["total_sales"].idxmin())

print("Highest-sales product:", product_summary["total_sales"].idxmax())
print("Lowest-sales product:", product_summary["total_sales"].idxmin())

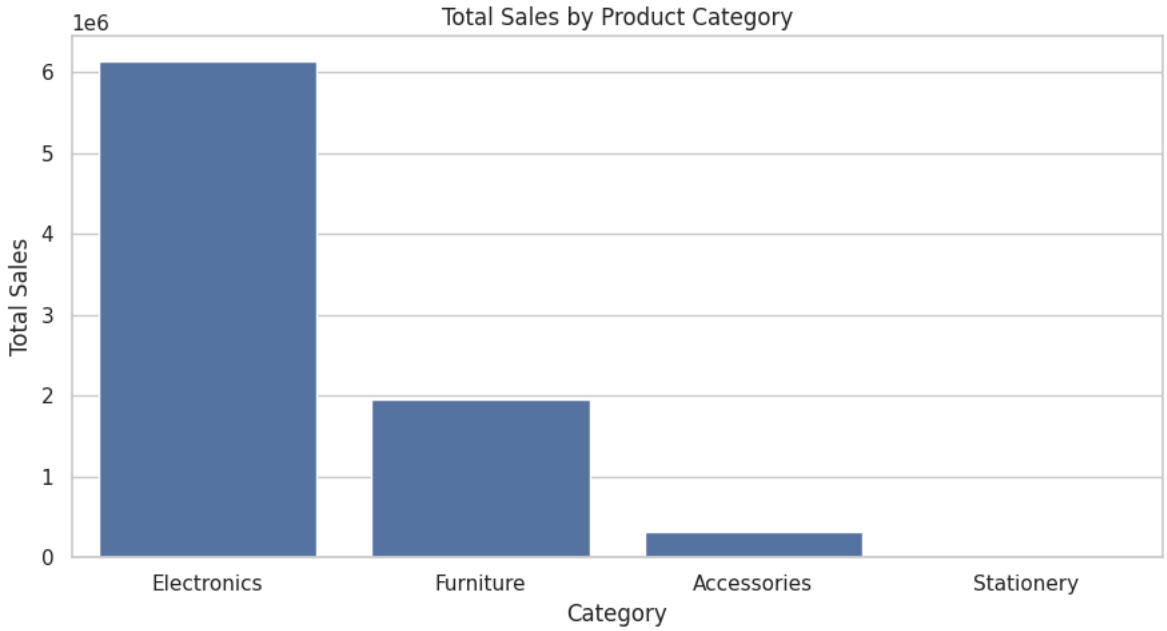
best_month = monthly_summary.loc[monthly_summary["total_sales"].idxmax()]
worst_month = monthly_summary.loc[monthly_summary["total_sales"].idxmin()]

print("Best month:", best_month["order_date"].strftime("%B %Y"),
      "Sales:", round(best_month["total_sales"], 2))
```

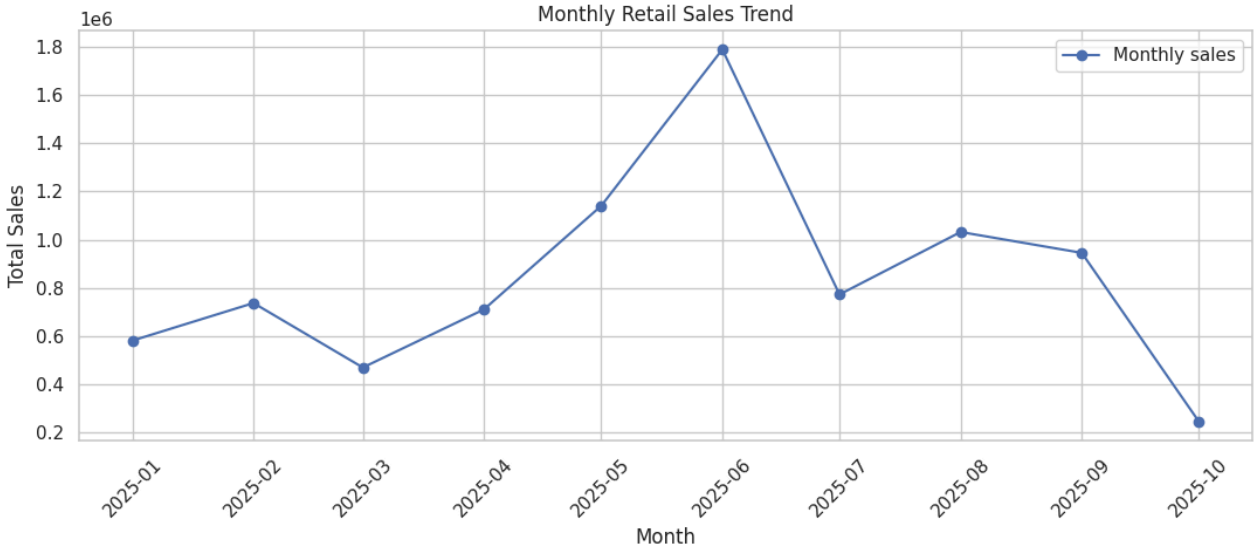
```
print("Lowest month:", worst_month["order_date"].strftime("%B %Y"),
      "Sales:", round(worst_month["total_sales"], 2))
```

```
Highest-sales category: Electronics
Lowest-sales category: Stationery
Highest-sales region: East
Lowest-sales region: South
Highest-sales product: Laptop
Lowest-sales product: Pen
Best month: June 2025 Sales: 1791350.0
Lowest month: October 2025 Sales: 242145.0
```

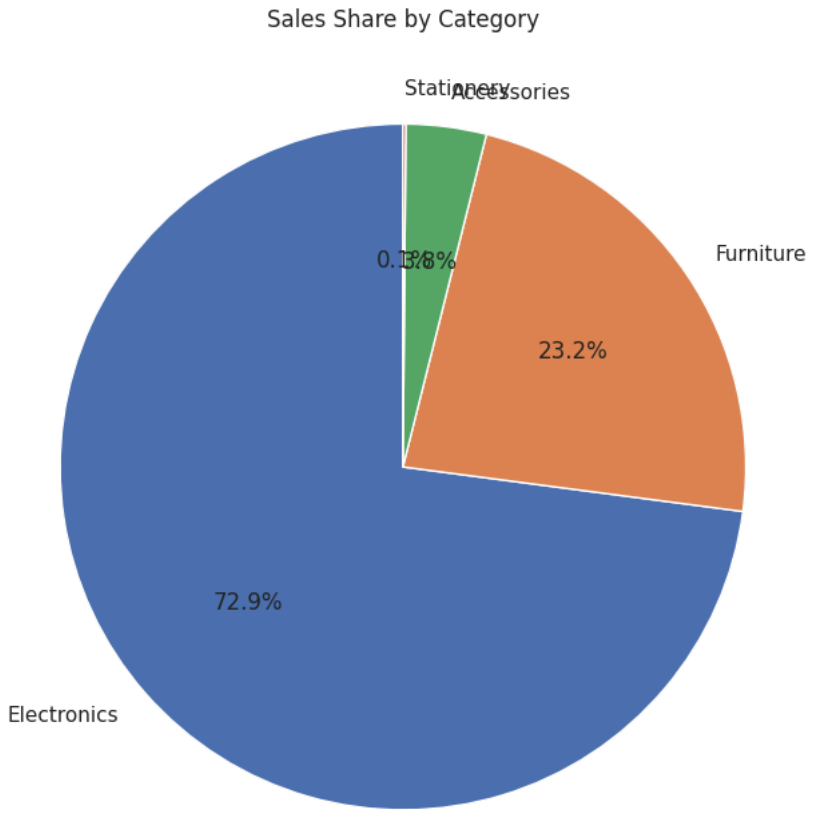
```
plt.figure(figsize=(9, 5))
sns.barplot(data=category_summary.reset_index(),
            x="category", y="total_sales")
plt.title("Total Sales by Product Category")
plt.xlabel("Category")
plt.ylabel("Total Sales")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "01_bar_chart.png", dpi=150)
plt.show()
```




```
plt.figure(figsize=(11, 5))
plt.plot(monthly_summary["order_date"],
         monthly_summary["total_sales"], marker="o",
         label="Monthly sales")
plt.title("Monthly Retail Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.legend()
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "02_line_chart.png", dpi=150)
plt.show()
```




```
plt.figure(figsize=(7, 7))
plt.pie(category_summary["total_sales"],
        labels=category_summary.index,
        autopct="%1.1f%%", startangle=90)
plt.title("Sales Share by Category")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "03_pie_chart.png", dpi=150)
plt.show()
```



```
plt.figure(figsize=(9, 5))
sns.histplot(data=clean_df, x="sales", bins=20, kde=True)
plt.title("Distribution of Sales per Order")
plt.xlabel("Sales Amount")
plt.ylabel("Number of Orders")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "04_histogram.png", dpi=150)
plt.show()
```

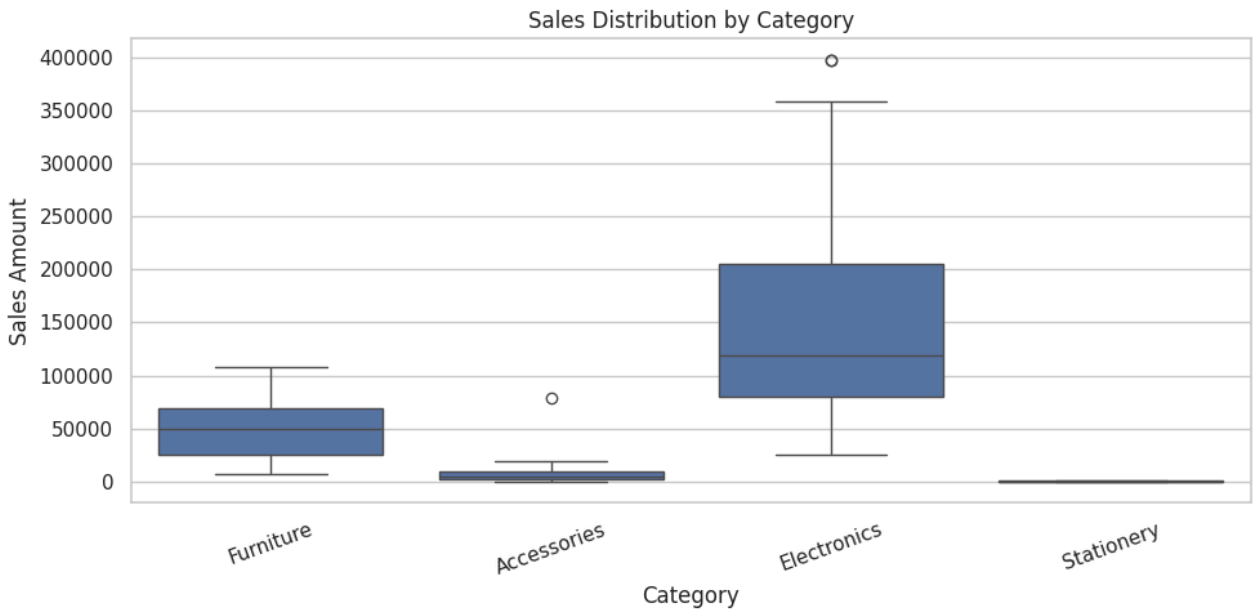



```
plt.figure(figsize=(9, 5))
sns.scatterplot(data=clean_df, x="quantity", y="sales",
                hue="category")
plt.title("Quantity vs Sales")
plt.xlabel("Quantity")
plt.ylabel("Sales Amount")
plt.legend(title="Category", bbox_to_anchor=(1.02, 1),
           loc="upper left")
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "05_scatter_plot.png", dpi=150)
plt.show()
```




```
plt.figure(figsize=(10, 5))
sns.boxplot(data=clean_df, x="category", y="sales")
plt.title("Sales Distribution by Category")
plt.xlabel("Category")
plt.ylabel("Sales Amount")
plt.xticks(rotation=20)
plt.tight_layout()
plt.savefig(OUTPUT_DIR / "06_box_plot.png", dpi=150)
```

```
plt.savefig(OUTPUT_DIR / "06_box_plot.png", dpi=150)
plt.show()
```




```
plt.figure(figsize=(11, 5))
plt.fill_between(monthly_summary["order_date"],
                 monthly_summary["total_sales"], alpha=0.4)
plt.plot(monthly_summary["order_date"],
         monthly_summary["total_sales"], marker="o")
plt.title("Monthly Sales Area Chart")
plt.xlabel("Month")
```